

ResearchLens

Comprehensive Engineering & Architecture Audit

Codebase: researchlens_project.zip

Audit date: 8 October 2026

Scope: architecture, execution pipeline, retrieval, evidence grounding, NLI, report generation, testing, security, deployment, and maintainability

Overall assessment: 7.2 / 10

The project is already a substantial AI research assistant rather than a simple RAG demo. Its strongest differentiator is the evidence-first design: retrieved papers are converted into passages, evidence is filtered, citations are validated, and the final report is checked before export. The main work remaining is correctness hardening, benchmark-driven evaluation, and removal of research-topic-specific shortcuts.

Audit conclusion: Do not rewrite the project. Stabilize the evidence/citation contract, make query planning domain-generic, improve retrieval evaluation, and then move selected components toward production architecture.

1. Executive Summary

ResearchLens has a strong modular architecture with a FastAPI backend, Streamlit interface, scholarly retrieval from multiple sources, metadata enrichment, reranking, PDF extraction, embeddings and FAISS retrieval, evidence verification, NLI-based answer verification, citation/report validation, PDF generation, security controls, benchmarks, and a relatively large test suite.

The implementation is strongest where it treats research generation as a traceability problem rather than a pure text-generation problem. The main technical concern is that some internal contracts do not fully match their names or documentation. In particular, claim-level evidence verification currently uses CrossEncoder relevance plus a verbatim source-match check; NLI is applied later at answer level. There is also a potentially unsafe cross-paper re-attribution path that can pair a passage from one paper with metadata from another paper.

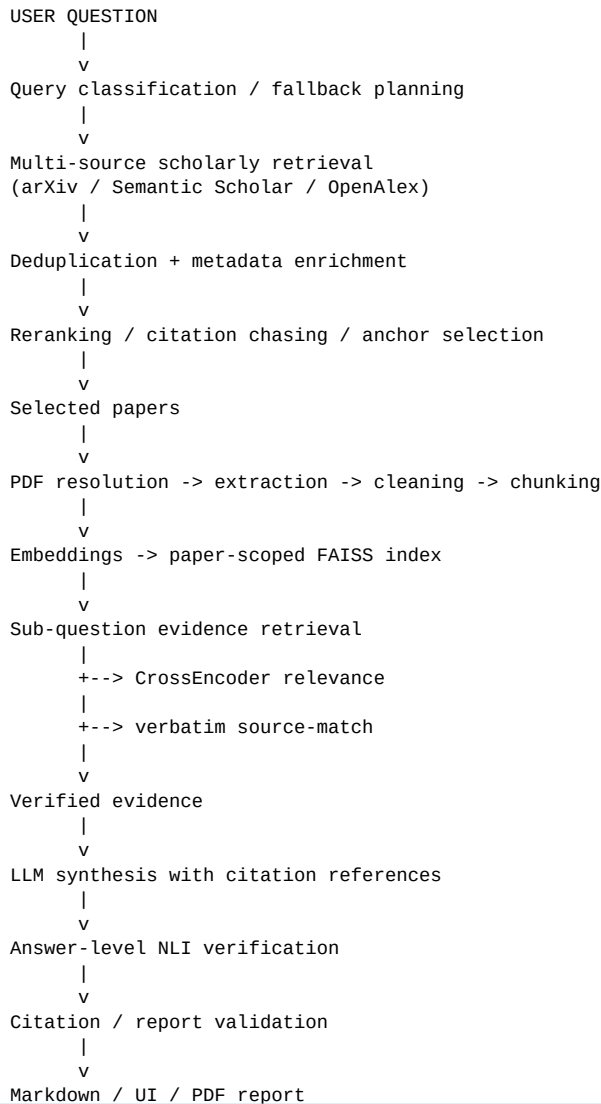
Area	Score	Audit judgment
Architecture	8.5/10	Good service separation and clear pipeline stages.
Retrieval	7.5/10	Good multi-source foundation; needs hybrid retrieval and measured recall.
RAG / vector search	8.0/10	Paper-scoped search is a strong correctness decision.
Evidence grounding	7.0/10	Strong concept; provenance re-attribution needs hardening.
NLI verification	7.0/10	Useful answer-level gate, but terminology should be clearer.
LLM synthesis	8.0/10	Citation-aware generation and validation are strong.
PDF/reporting	8.5/10	Exporter and regression coverage are comparatively mature.
Testing	6.5/10	Targeted tests pass, but full collection currently fails on dependency import.
Production readiness	5.5/10	BackgroundTasks, SQLite, and in-memory job indexes limit scale.
Documentation consistency	5.5/10	Model/config descriptions have drifted.

Highest-priority findings:

- **P0:** Fix cross-paper evidence attribution so a passage can never be silently re-labeled as belonging to another paper without re-retrieving and re-verifying that passage from the target paper.
- **P1:** Remove hardcoded LLM-inference concepts, queries, titles, and sub-questions from the generic fallback planner.
- **P1:** Align NLI terminology and architecture documentation with the actual implementation.
- **P1:** Remove the hardcoded OpenAlex email and make domain/field filtering explicitly configurable.
- **P1:** Correct the PDF regression test that still expects duplicated summary/findings text.
- **P2:** Replace early PDF/page/passages caps with extract-all-then-rank behavior and add section-aware chunking.
- **P2:** Build a labeled retrieval/evidence benchmark and report Recall@K, MRR, NDCG, evidence precision/recall, citation precision/recall, unsupported-claim rate, and latency.

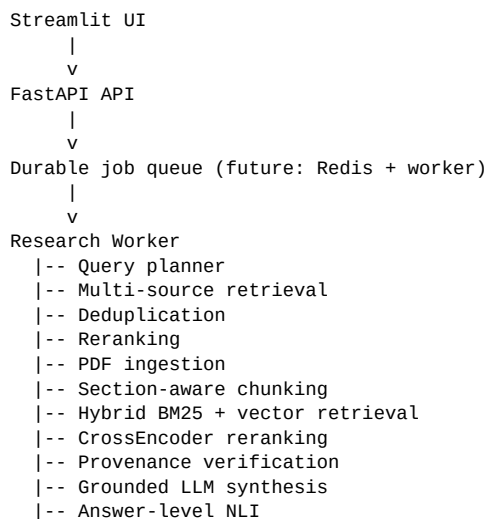
2. Current System Architecture

The project can be understood as the following execution graph. The diagram is intentionally simplified to show responsibilities rather than every helper function.



This is a good architecture for an evidence-grounded research assistant. The major improvement is to make the boundaries explicit: retrieval relevance, evidence provenance, NLI entailment, and citation integrity should be separate concepts with separate outputs.

Recommended target architecture



```
|-- Citation/report validation  
'-- Report generation
```

3. End-to-End Pipeline Audit

Stage	Current implementation	Assessment	Recommended change
1. Planning	Question type + deterministic fallback planner	Good baseline, but topic-specific hardcoding exists.	Use dynamic facets and generic query generation.
2. Retrieval	arXiv / Semantic Scholar / OpenAlex	Strong multi-source coverage.	Add retrieval observability and benchmark recall.
3. Dedup	DOI/arXiv/S2/OpenAlex/title-style identifiers	Good and necessary.	Centralize a canonical PaperIdentity object.
4. Ranking	Semantic reranking + citation/title signals	Useful but thresholds are not calibrated.	Calibrate reranker scores and measure NDCG/MRR.
5. PDF ingestion	PyMuPDF, header/footer filtering, reference cutoff	Good preprocessing foundation.	Do not truncate before ranking; preserve all useful pages.
6. Chunking	3-sentence windows, 1-sentence overlap	Simple and deterministic.	Use section/paragraph/token-aware chunks.
7. Vector retrieval	Paper-scoped FAISS	One of the strongest design choices.	Keep for MVP; add hybrid retrieval later.
8. Evidence	Sentence candidates + relevance + verbatim source match	Strong provenance concept, but attribution risk.	Re-retrieve and re-verify after any paper reassignment.
9. Synthesis	Citation-aware LLM report generation	Strong.	Keep; expose claim-to-evidence lineage.
10. NLI	Answer-level verification	Useful safety gate.	Rename/clarify as answer-level NLI.
11. Validation	Citation/report validation	Strong.	Add quantitative benchmark metrics.
12. Export	Streamlit/Markdown/PDF	Mature relative to other parts.	Keep stable; avoid unnecessary refactor.

Pipeline design principle

The most important architectural rule going forward should be: never let a generated claim acquire a paper identity merely because a matching author name was found. Paper identity must be derived from the exact passage that supports the claim, and any re-attribution must trigger a new retrieval and source-match check.

4. Line-Level Findings and Code Audit

The following findings are grounded in the uploaded source. Line numbers refer to the extracted ZIP as audited on 8 October 2026.

4.1 P0 - Cross-paper evidence attribution risk

File: backend/services/evidence.py, approximately lines 301-336 and 364-385.

```
301 # Author attribution alignment check
303 ext_author_matches = re.findall(...)
304 target_rec = rec
...
317 if matched_other_rec is not None:
318     # Re-attribute claim to the actual author's paper
319     target_rec = matched_other_rec
...
330 candidate_claims.append({
333     "record": target_rec,
334     "passage_text": passage_text,
...
367 record = item["record"]
368 p_text = item["passage_text"]
373 is_source_match = check_verbatim_source_match(claim_text, p_text)
```

Why this matters: the code can set record to a different paper while keeping passage_text from the original paper. The later source-match check verifies the claim against the original passage, not necessarily against the newly assigned paper. This creates a possible citation/provenance mismatch.

Required fix: if an external author is detected, re-retrieve the claim from the candidate target paper, then verify the new passage and only then change the record. If no verified target passage exists, reject the candidate.

4.2 P1 - Query planner contains topic-specific hardcoding

File: backend/services/query_planner.py, lines 63-108 and the fallback branch around line 130 onward.

```
LLM_INFERENCE_CONCEPTS = [...]
LLM_INFERENCE_FOCUSED_QUERIES = [...]
LLM_INFERENCE_TITLES = [...]
LLM_INFERENCE_SUB_QUESTIONS = [...]

def is_llm_inference_optimization_query(question: str) -> bool:
    ...
```

This means a generic research assistant has a hidden path specifically for LLM inference optimization. It is useful for a demo question but weakens the claim of autonomous, domain-general research planning.

Fix: replace the topic-specific branch with dynamic facets generated from the question. A fallback planner should extract entities, concepts, constraints, time range, and comparison dimensions without injecting known papers.

4.3 P1 - NLI terminology does not match the evidence implementation

File: backend/services/evidence.py, lines 348-358.

```
348 # Compute cross-encoder relevance scores
350 rel_pairs = [[c["sub_q"], c["sentence"]] ...]
352 rel_scores = sigmoid(CrossEncoder.predict(...))

356 # Plain Claim-Level Check: "source match" + Relevance Gate
357 # Replaces the NLI call on claims with a plain check
```

The implementation is actually performing **retrieval relevance + verbatim source matching** at the evidence stage. NLI is a later answer-level verification layer. This is a defensible design, but the names and documentation should say so explicitly.

4.4 P1 - Hardcoded OpenAlex contact email and fixed field filter

File: backend/config.py, lines 42-48.

```
OPENALEX_EMAIL: str = "rohitkumar700as@gmail.com"
OPENALEX_API_KEY: str = ""
OPENALEX_FIELD_FILTER: str = "topics.field.id:17"
DEFAULT_FIELDS_OF_STUDY: str = "Computer Science"
```

The email should come only from environment configuration. The default OpenAlex field filter also biases retrieval toward a specific domain even though the UI presents a general research workflow.

4.5 P2 - PDF extraction is capped before retrieval

File: backend/services/pdf_extractor.py, around lines 140-144 and 240 onward.

```
total_pages = min(len(doc), 30) # max 30 pages
...
# 3-sentence windows with 1-sentence overlap
...
max_passages = getattr(settings, "MAX_PASSAGES_PER_PAPER", 25)
```

This protects memory in constrained environments, but it can discard evidence from long papers before semantic retrieval has a chance to rank it. A better strategy is extract-all, clean, index, then rank/select.

4.6 P1 - PDF regression test still encodes duplicate summary/findings behavior

File: backend/tests/test_pdf_report.py, lines 59-72.

```
SUMMARY = ("Research coverage is currently limited. ...")

def test_extracted_text_has_no_repeats():
    ...
    assert text.count("Research coverage is currently limited") == 2
```

The test explicitly expects the same substantive sentence twice because it is placed in both executive summary and findings. If the implementation is intended to prevent duplicate report content, this test should assert one occurrence or use a stronger duplicate-content rule.

5. Additional Engineering Findings

5.1 Ranking score calibration

The ranker uses CrossEncoder scores and converts raw scores with a sigmoid-like transformation. That makes the value numerically convenient, but it should not be interpreted as a calibrated probability without validation. A threshold such as 0.30 should be selected from labeled data, not from the transformed score alone.

Recommendation: create a relevance dataset with question/paper labels and report Recall@K, Precision@K, MRR, NDCG@K, and threshold calibration.

5.2 Streamlit and FastAPI both own pipeline execution

backend/routers/research.py schedules the research workflow with FastAPI BackgroundTasks, while streamlit_app.py also contains its own worker path. This duplicates orchestration responsibility.

```
FastAPI:
background_tasks.add_task(_run_pipeline_with_new_session, job_id)

Streamlit:
def _run_pipeline_worker(job_id: str) -> None:
    ...
```

MVP: acceptable. **Production target:** Streamlit should call FastAPI only; FastAPI should enqueue work; a durable worker should execute the pipeline.

5.3 BackgroundTasks are not a durable job system

FastAPI BackgroundTasks are suitable for a demo or small deployment, but a process restart can lose work. For production, use a durable queue such as Redis-backed workers, and persist job state transitions in the database.

5.4 Data model is convenient but JSON-heavy

Large report/debug/pipeline/statistics blobs and JSON-encoded passages are easy for an MVP but become difficult to query and update as the system grows. A future production schema should normalize papers, passages, claims, evidence, citations, and reports.

5.5 Security is reasonable for an MVP

The project includes API key/bearer authentication and rate limiting. The main production concern is trust around client IP headers such as X-Forwarded-For; those should only be accepted from a trusted reverse proxy.

5.6 Documentation/configuration drift

The repository contains multiple configuration examples and documentation files. Model names and architecture descriptions can drift from runtime configuration. Establish one source of truth in config.py and make the documentation describe that configuration rather than duplicating model identifiers in multiple places.

5.7 Strong design choice - paper-scoped vector search

The vector store supports paper-scoped retrieval rather than searching a global FAISS index and filtering afterward. This is a correctness-oriented decision: it prevents a relevant passage from the requested paper from being crowded out by passages from other papers.

5.8 Strong design choice - citation reference mapping

backend/services/local_llm_service.py explicitly builds a reference index and assigns citation badge numbers from that mapping. This reduces the common RAG failure where citation badges and reference-list positions disagree.

6. Testing and Verification Audit

Two test levels were executed against the extracted ZIP.

Verification	Result	Interpretation
python -m compileall -q backend	PASS	Backend source compiles successfully.
Targeted pytest suite	40 passed, 2 skipped	Core selected tests pass in the audit environment.
Full pytest collection	FAIL: 9 collection errors	Import-time failure because transformers is not installed in the audit environment.

The full-suite failure is primarily an environment/dependency issue rather than proof of functional failure. The repository's backend requirements do include transformers, torch, sentence-transformers, and faiss-cpu. However, the test suite is not fully self-contained if collection requires heavyweight ML dependencies before tests can decide whether they need them.

Recommended test pyramid:

```
Layer 1 - Pure unit tests
    text utilities, query normalization, schemas, scoring helpers

Layer 2 - Mocked integration tests
    retrieval clients, evidence pipeline, report validation

Layer 3 - Real-model smoke tests
    embeddings, CrossEncoder, NLI on tiny fixtures

Layer 4 - End-to-end benchmark
    fixed research questions -> papers -> evidence -> report
```

The project should also distinguish correctness tests from quality benchmarks. Passing tests proves deterministic behavior; it does not prove that retrieval is relevant or that generated citations are useful.

7. Evaluation Plan - Turn ResearchLens into a Measurable System

The biggest missing artifact is a labeled benchmark. Build a small, high-quality benchmark before attempting a major architecture rewrite.

Suggested benchmark design

Dataset component	Suggested size	Purpose
Research questions	100	Cover factual, comparative, survey, and technical questions.
Relevant papers per question	3-10	Measure retrieval recall.
Relevant passages	2-5 per paper	Measure evidence retrieval.
Claim support labels	5-15 claims/question	Measure citation/evidence grounding.
Difficulty tags	per question	Short papers, long papers, conflicting evidence, ambiguous terms, etc.

Metrics to report

Metric	What it answers
Recall@5 / Recall@10	Did the system retrieve the papers needed to answer the question?
MRR	How early does the first relevant paper appear?
NDCG@10	Is ranking quality good across multiple relevant papers?
Evidence Precision	How much retrieved evidence is actually useful/supporting?
Evidence Recall	How much of the labeled supporting evidence was recovered?
Citation Precision	How many citations actually support their claims?
Citation Recall	How many important claims are adequately cited?
Unsupported Claim Rate	How often does the report assert something unsupported?
Source Diversity	Does the answer rely on a narrow or broad literature base?
End-to-end latency	How long does a complete research run take?
PDF success rate	How often can full text be resolved and parsed?

Target differentiator: expose a claim-to-evidence trace in the UI. For every generated statement, show the paper, page, section, passage, source-match status, relevance score, NLI score, and final citation badge.

8. Recommended Target Architecture

```
researchlens/  
  frontend/  
    streamlit_app.py  
  
  backend/  
    api/  
    core/  
    models/  
    retrieval/  
      arxiv.py  
      semantic_scholar.py  
      openalex.py  
      deduplication.py  
      reranker.py  
    ingestion/  
      pdf_downloader.py  
      pdf_parser.py  
      chunker.py  
    retrieval_engine/  
      bm25.py  
      vector.py  
      hybrid.py  
    evidence/  
      extractor.py  
      provenance.py  
      nli.py  
    synthesis/  
      planner.py  
      writer.py  
      citation_validator.py  
    reports/  
      validator.py  
      markdown.py  
      pdf.py  
    workflow/  
      research_pipeline.py  
  
  evaluation/  
    dataset.json  
    benchmark.py  
    metrics.py  
  
  tests/  
    unit/  
    integration/  
    e2e/  
  
  docs/  
    architecture.md  
    methodology.md  
    evaluation.md
```

The target structure separates ingestion, retrieval, evidence, synthesis, reporting, and evaluation. Do not refactor the repository into this structure all at once; use it as the destination while making incremental, test-backed changes.

9. Implementation Roadmap

Phase	Work	Priority	Exit criterion
Phase 1	Fix cross-paper provenance; correct PDF duplicate regression; remove hardcoded email.	P0/P1	No known citation identity mismatch; tests updated.
Phase 2	Remove topic-specific planner shortcuts; introduce dynamic facets; align docs/config.	P1	Same question types work without special topic branch.
Phase 3	Extract all PDF pages; section-aware chunks; preserve page/section/offset metadata.	P2	Long papers retain relevant evidence.
Phase 4	Add BM25 + vector hybrid retrieval and calibrated reranker thresholds.	P2	Benchmark shows measurable Recall/NDCG improvement.
Phase 5	Build 100-question benchmark and evidence/citation evaluation.	P2	Quality metrics are reproducible.
Phase 6	Expose claim-to-evidence trace in UI and report.	P2	Every report claim can be traced to evidence.
Phase 7	Move to durable queue/PostgreSQL only if deployment needs it.	P3	Production deployment survives restarts and concurrency.

10. Priority Matrix

ID	Severity	Finding	File / area	Action
RL-001	P0	Cross-paper record can diverge from passage provenance.	evidence.py	Re-retrieve and re-verify target paper before re-attribution.
RL-002	P1	Topic-specific LLM inference hardcoding in generic planner.	query_planner.py	Replace with dynamic facets.
RL-003	P1	NLI terminology does not describe claim-level check.	evidence.py / nli_verifier.py	Separate provenance verification from answer NLI.
RL-004	P1	Hardcoded OpenAlex email.	config.py	Environment-only configuration.
RL-005	P1	OpenAlex field filter can bias general research.	config.py / retrieval	Make domain filter explicit/dynamic.
RL-006	P1	PDF regression test expects duplicate text.	test_pdf_report.py	Change to semantic duplicate check.
RL-007	P2	PDF extraction capped before retrieval.	pdf_extractor.py	Extract all, rank later.
RL-008	P2	Chunking is sentence-window only.	pdf_extractor.py	Section/paragraph/token-aware chunks.
RL-009	P2	CrossEncoder scores treated like probabilities.	ranker/evidence	Calibrate thresholds on labeled data.
RL-010	P2	No complete retrieval/evidence quality benchmark.	evaluation	Create labeled benchmark and metrics.
RL-011	P3	Streamlit and FastAPI both execute pipeline.	streamlit_app.py / router	Consolidate orchestration behind API/worker.
RL-012	P3	BackgroundTasks not durable.	FastAPI	Move to queue worker for production.
RL-013	P3	JSON-heavy persistence.	database/models	Normalize after benchmark and scale needs.

11. How to Present ResearchLens

For a resume, interview, or portfolio, avoid describing the project as only a 'RAG chatbot'. The stronger positioning is:

ResearchLens - Evidence-Grounded Autonomous Research Assistant

- Multi-source scholarly retrieval across arXiv, Semantic Scholar, and OpenAlex.
- Paper-scoped vector retrieval to prevent cross-paper evidence contamination.
- CrossEncoder reranking and citation-aware paper selection.
- PDF extraction with page/section-aware evidence tracking.
- Evidence-first report generation with citation validation.
- Answer-level NLI verification and report-quality gates.
- Streamlit research UI with downloadable PDF reports.
- Benchmarkable retrieval, evidence, citation, and latency metrics.

For interviews, the most defensible technical story is:

Problem:

LLM-generated research reports can hallucinate claims or citations.

Design:

Retrieve scholarly papers -> extract evidence -> verify provenance

-> synthesize only from verified evidence -> validate citations

-> verify generated answer sentences -> export report.

Key engineering insight:

Retrieval relevance, evidence provenance, NLI entailment,
and citation integrity are different verification problems.

Once the P0/P1 issues are fixed and the benchmark exists, this project can be presented as a substantially stronger AI engineering project than a standard LangChain/RAG demo.

12. Final Verdict

Current state: strong prototype / advanced portfolio project with a credible research pipeline.

Not yet: fully validated autonomous research system or production-grade distributed service.

Do not rewrite: the architecture already has the right major building blocks.

Fix first: evidence provenance, planner hardcoding, terminology/config drift, and regression expectations.

Then measure: retrieval quality, evidence quality, citation quality, unsupported claims, and latency.

Then scale: hybrid retrieval, durable workers, PostgreSQL, and persistent vector infrastructure only where justified.

Recommended rating after P0/P1 fixes + benchmark

Dimension	Current	Realistic after fixes
Correctness / provenance	6.5	8.5+
Retrieval quality	7.5	8.5+
Evaluation maturity	5.5	8.5+
Architecture	8.5	9.0
Portfolio strength	8.0	9.0+

Bottom line: ResearchLens is worth continuing. The highest-value work is not adding more AI features; it is making the existing evidence chain formally correct, measurable, and explainable.

Appendix A - Audit methodology

The ZIP was extracted and inspected as a source tree. Key service files, configuration, tests, and orchestration paths were reviewed at line level. The backend was compiled with Python compileall. A targeted pytest set was executed successfully (40 passed, 2 skipped). A full pytest collection was also attempted; 9 test modules failed during collection because the audit environment did not have the heavyweight transformers dependency installed. This report therefore distinguishes source-code findings from environment-dependent test-collection failures.